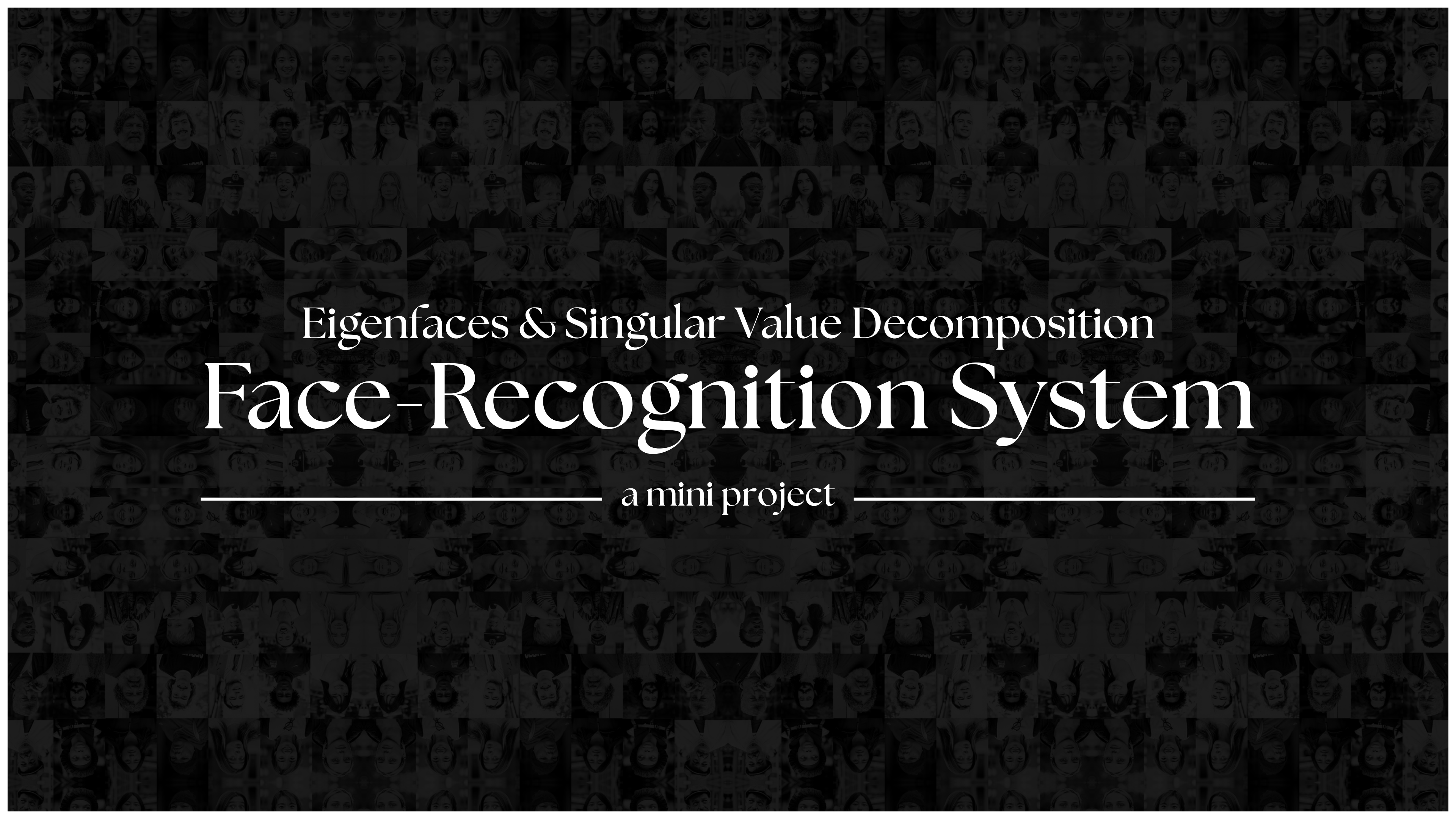




Eigenfaces & Singular Value Decomposition Face-Recognition System

— a mini project —

Jonathan Delen Hosea 10122054

Dzubyan Ilman Ramadhan 10122010

Rachma Nurhidayah 10122037

Adhityo Rafael Andy Sigit 10122020

Hendri Saputra 10122024

MASALAH: DATA WAJAH BERDIMENSI TINGGI



Satu foto memuat
 $92 \times 112 = 10304$ nilai pixel

Tantangan terhadap
sistem pengenalan wajah:

- tidak akurat karena latar belakang, pencahayaan, serta posisi
- lambat serta boros memori.

SOLUSI: MENEMUKAN “POLA DASAR” WAJAH



SVD



Tujuan: Mencari representasi ringkas **Metode:** Eigenfaces melalui SVD

ALUR KERJA IMPLEMENTASI

**DATASET
WAJAH**

**VEKTORISASI
& HITUNG
RATA-RATA**

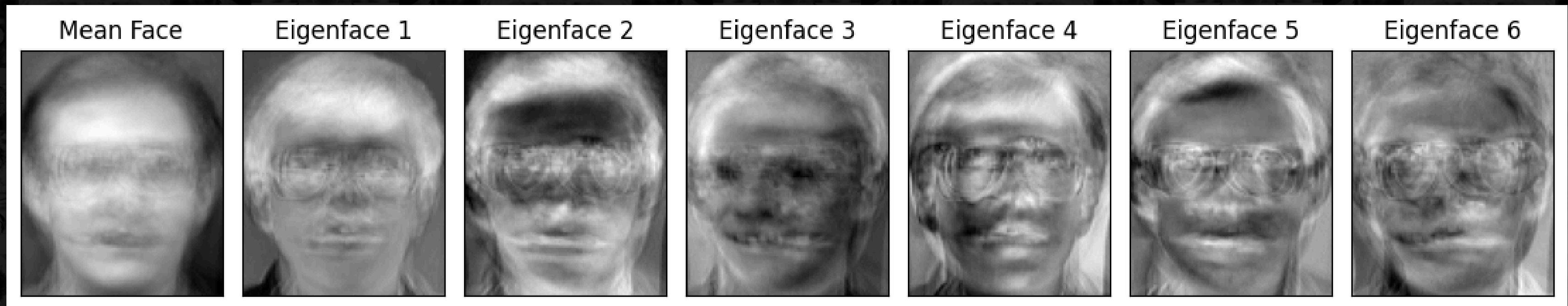
**DAPATKAN
EIGENFACES
(BASIS BARU)**

**TERAPKAN
SVD**

**PROYEKSI KE
RUANG WAJAH
($k = 10$)**

**KLASIFIKASI
& HITUNG
AKURASI**

HASIL: GALERI EIGENFACES



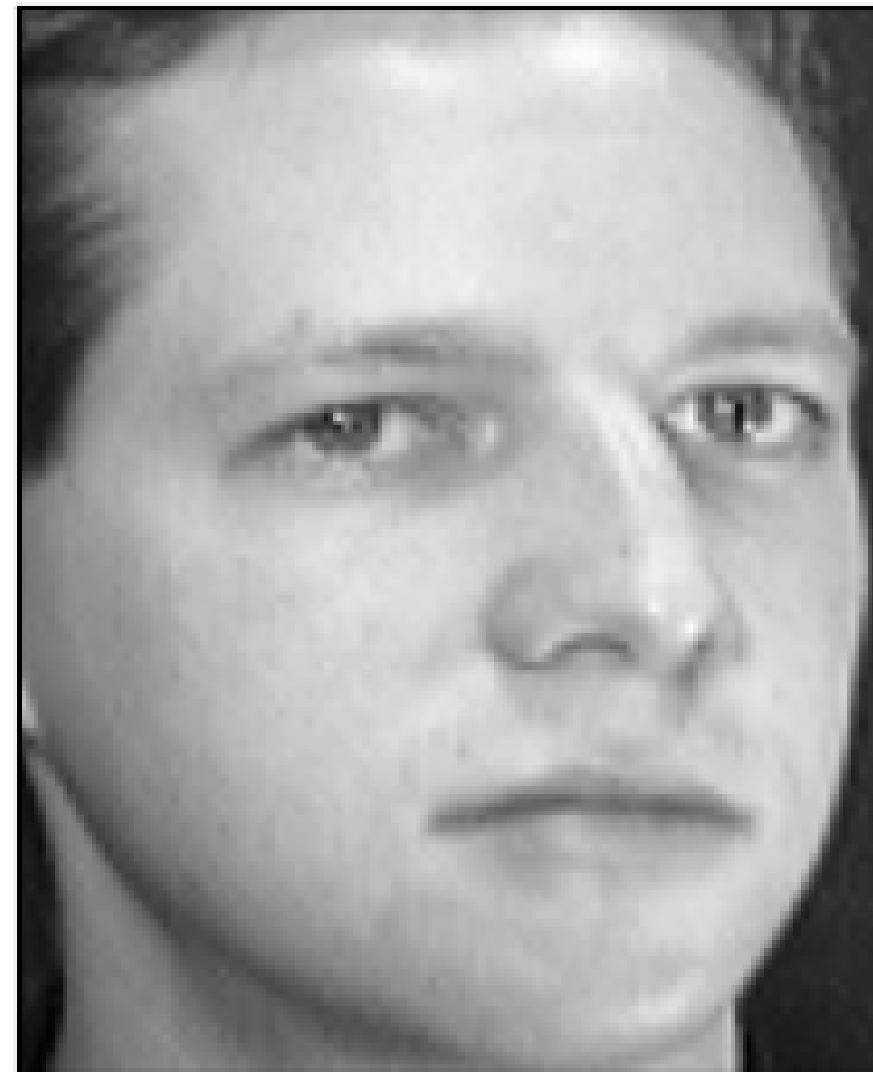
Wajah Rata-rata (kiri) dan
6 komponen variasi utama (*eigenfaces*) dari dataset

HASIL UTAMA: AKURASI DAN REKONSTRUKSI

Hasil Akhir Akurasi Pengenalan Wajah

k	Akurasi (%)
10	100.00
20	100.00

Gambar Asli (Subjek 1)



Rekonstruksi (k=10)



KAITAN DENGAN KONSEP ALJABAR LINEAR

- ***Vektor dan Ruang Vektor***

Gambar diperlakukan sebagai vektor

- ***Singular Value Decomposition (SVD)***

Fungsi `numpy.linalg.svd` dipakai untuk menemukan basis baru (*eigenfaces*)

- ***Proyeksi Ortogonal***

Rumus $Uk.T @ (x - \mu)$ dipakai untuk mereduksi dimensi

- ***Norma dan Jarak Euclidean***

Fungsi `numpy.linalg.norm` dipakai untuk klasifikasi.

KAITAN DENGAN KONSEP ALJABAR LINEAR

- ***Vektor dan Ruang Vektor*** <section Dataset, code ke-3>

```
# Fungsi berikut memuat gambar dari folder, mengubahnya menjadi vektor,  
# dan membaginya menjadi set training dan testing.  
def load_data():  
    print("Memuat data gambar...")  
    train_vectors = []  
    test_vectors = []  
    train_labels = []  
    test_labels = []
```

- ***Singular Value Decomposition (SVD)*** <section SVD>

```
# LAKUKAN "THIN SVD"  
# Gunakan SVD pada matriks data terpusat  
# full_matrices=False akan menjalankan "thin" SVD yang lebih efisien  
# Kolom-kolom dari matriks U adalah Eigenfaces yang kita cari  
  
U, Sigma, Vt = np.linalg.svd(X_centered, full_matrices=False)
```

KAITAN DENGAN KONSEP ALJABAR LINEAR

- ***Proyeksi Ortogonal*** <section Accuracy Table>

```
# 2. Proyeksikan data training dan testing ke "face space" k-dimensi
# Rumus:  $z = U_k.T * (x - \mu)$ 
Z_train = Uk.T @ X_centered # Data training sudah terpusat
Z_test = Uk.T @ (X_test - mean_face)
```

- ***Norma dan Jarak Euclidean*** <section Accuracy Table>

```
# 3. Klasifikasi dengan Nearest Neighbour
correct_predictions = 0
# Loop melalui setiap gambar testing
for i in range(Z_test.shape[1]):
    test_coord = Z_test[:, i]

    # Hitung jarak Euklides dari gambar tes ini ke seluruh gambar training
    distances = np.linalg.norm(Z_train - test_coord.reshape(-1, 1), axis=0)
```

KESIMPULAN

- SVD **berhasil** menemukan representasi wajah yang **efektif dan ringkas**
- Akurasi **100%** tercapai pada ruang berdimensi **rendah**, membuktikan kegunaan metode ini
- Jumlah k yang **kecil** sudah cukup untuk menghasilkan performa yang **maksimal** pada dataset ini.



THANK YOU